

NutriGuard AI

Event-Driven Multi-Agent Health & Nutrition Tracker

Project Design Document

Primary Type: Recommender | Agentic AI

June 2025

Table of Contents

1. Project Overview & Problem Statement

2. Market Research & Opportunity

3. Proposed Solution

4. Project Type Classification

5. High-Level Architecture

6. Tech Stack & Azure Services

7. Agent Design

8. Data Sources & Schema

9. Guardrails & Safety

10. Observability & Monitoring

11. Success Metrics

12. Scope & Out-of-Scope

1. Project Overview & Problem Statement

1.1 Project Name

NutriGuard AI: Event-Driven Multi-Agent Health & Nutrition Tracker

1.2 Target Users

NutriGuard AI is designed for the following user segments:

- Health-conscious individuals tracking diet improvements
- Working professionals with limited time for detailed meal logging
- Students managing nutrition on a budget
- Fitness beginners with specific body composition goals
- Individuals with basic health conditions (iron deficiency, Vitamin D/B12 deficiency)

1.3 The Pain Point

Most users do not know whether their daily meals align with their health goal or condition.

Examples of invisible risks:

- A user with iron deficiency drinks tea with every meal — not knowing it may affect iron absorption.
- A fat-loss user eats mostly carbohydrate-heavy meals and misses daily protein targets.
- A vegetarian with B12 deficiency has no fortified food source in their routine.

Current market tools fail to address this because:

- Standard diet apps require exact gram-level input — too tedious for everyday users.
- Generic AI chatbots give ungrounded advice not tied to trusted nutrition knowledge.
- No simple tool combines natural language meal input, health condition context, and safe recommendations together.

1.4 Final Problem Statement

Problem Statement

For health-conscious users with nutrition goals or basic health conditions, who need simple and safe feedback on daily meals without manually tracking calories, we build NutriGuard AI — a multi-agent nutrition risk and recommendation assistant that provides meal analysis, risk flags, and daily improvement suggestions, measured by risk flag accuracy, JSON validity, safety compliance, and report generation time. Unlike generic calorie trackers or ungrounded chatbot advice, NutriGuard AI uses agents to reason over meal logs, retrieve trusted nutrition knowledge, and act by generating safe personalized reports.

2. Market Research & Opportunity

2.1 Market Size

The global digital health and nutrition tracking market is undergoing rapid growth, driven by increasing awareness of preventive health and chronic disease management.

Segment	Market Size (2024)	Projected (2029)
Global Digital Health Market	USD 280 billion	USD 660 billion (CAGR ~18%)
Nutrition & Diet Apps Market	USD 6.4 billion	USD 13.6 billion (CAGR ~16%)
AI in Healthcare Market	USD 22 billion	USD 120 billion (CAGR ~40%)
India Digital Health (Target)	USD 5 billion	USD 21 billion (CAGR ~33%)

Sources: MarketsandMarkets, Grand View Research, NITI Aayog Digital Health Reports (2024).

2.2 Competitive Landscape

Existing tools in the nutrition and diet tracking space fall into four broad categories:

Tool / Category	What They Do	Key Gap
MyFitnessPal / Cronometer	Calorie & macro tracking via barcode scans and manual entry	Requires exact gram input; no health condition intelligence
Noom / Lifesum	Habit coaching apps with diet plans	Generic plans; no real-time meal risk analysis
ChatGPT / Gemini (raw)	General nutrition Q&A via chat	No trusted knowledge base; no structured risk flags; no personalization
Healthifyme (India)	Diet tracking + dietitian chat	Requires premium subscription; no agentic reasoning
NutriGuard AI	Multi-agent meal analysis with condition-aware risk flags	Fills the gap — natural language + health context + safe recommendations

2.3 Key Differentiators

- Natural language meal input: No gram-level data entry needed.
- Condition-aware intelligence: Personalized to iron deficiency, B12, Vitamin D, fat loss, muscle gain, and vegetarian diet needs.
- Multi-agent reasoning: Three specialized agents handle extraction, risk assessment, and report generation.
- Trusted knowledge base: Risk flags are grounded in curated nutrition knowledge via Azure AI Search — not hallucinated.
- Safe-by-design: Built-in guardrails prevent diagnosis, medication advice, or supplement dosage recommendations.
- India-first use cases: Poha, dal rice, sabzi, and Indian food contexts handled natively.

2.4 Target User Personas

Persona	Profile	Key Need
Priya, 28 — Working Professional	Iron deficiency; eats dal rice + tea daily; no time for detailed logging	Know if her meals are helping or hurting her iron levels
Rahul, 22 — Fitness Beginner	Fat loss goal; eats poha, biscuits, and rice; misses protein	Understand if daily meals are aligned with fat loss
Anita, 45 — Homemaker	Vegetarian; Vitamin D deficiency; follows family cooking	Find out if her diet has adequate Vitamin D and B12 sources
Dev, 19 — Engineering Student	Irregular meals; skips breakfast; low energy	Get simple suggestions to improve energy and nutrition balance

2.5 Market Opportunity Summary

Opportunity

Over 500 million people in India alone are estimated to have some form of micronutrient deficiency (iron, B12, or Vitamin D). Most have smartphones but no accessible, intelligent nutrition guidance tool built for their food culture and language. NutriGuard AI addresses this gap by combining agentic AI, trusted retrieval, and safe recommendation logic.

3. Proposed Solution

3.1 What NutriGuard AI Does

NutriGuard AI is a multi-agent GenAI system that reads a user's daily meal log, checks it against their health goal and health condition, and generates safe nutrition risk flags with simple, actionable recommendations.

The system works in three steps:

- Step 1 — Understand the meal: Extract structured food information from simple natural language (e.g., 'I ate poha with tea').
- Step 2 — Identify risks: Cross-check the meal with the user's health condition and goal using trusted nutrition knowledge.
- Step 3 — Generate a report: Produce a safe daily nutrition report with risk flags and improvement suggestions.

3.2 Sample Input and Output

Sample User Input

User Profile

Health condition: Iron deficiency | Goal: Fat loss | Diet: Vegetarian | Meal: 'I ate poha with tea and missed my iron tablet.'

Sample Report Output

NutriGuard AI — Daily Nutrition Report

Today's breakfast was carbohydrate-heavy with low protein.

Risk Flags:

- Breakfast has low protein — may not support fat loss.
- Tea near meals may need attention for iron deficiency.
- Iron supplement missed today — note for tomorrow.

Suggestions:

- Add sprouts, curd, paneer, tofu, chana, or soy chunks to breakfast.
- Try separating tea from iron-rich meals by 1–2 hours.

Safety Note: This is general nutrition guidance only. Follow your doctor's advice for supplements or medical conditions.

4. Project Type Classification

4.1 Primary Type: Recommender

NutriGuard AI is primarily a Recommender system. Its core output is not a factual answer — it is a ranked set of safe, personalized next-best-action suggestions for improving the user's daily nutrition.

Input	Output
Meal text + user health profile	Risk flags + improvement suggestions
Condition: iron deficiency + tea in meal	Flag: tea may affect iron absorption
Goal: fat loss + low-protein breakfast	Suggestion: add sprouts, curd, or paneer
Missed supplement note	Reminder + safety note to follow doctor's advice

4.2 Agentic Angle

NutriGuard AI has a strong agentic design. Three specialized agents handle distinct reasoning tasks:

Agent	Type	Task
Meal Analyzer Agent	Reasoning	Parse natural language meal text; extract food items, categories, and missing nutrients
Health Risk & Supplement Agent	Reasoning + Retrieval	Retrieve trusted nutrition knowledge from Azure AI Search; generate condition-aware risk flags
Report Agent	Acting	Compose the final daily nutrition report with safe language, suggestions, and safety disclaimers

4.3 Secondary Project Types

- Chatbot (minor): Users enter meals in natural, conversational language — the system understands informal phrasing.
- Summariser (minor): The Report Agent summarises the daily nutrition pattern from multiple meal logs into a single cohesive report.

5. High-Level Architecture

5.0 Architecture Diagram

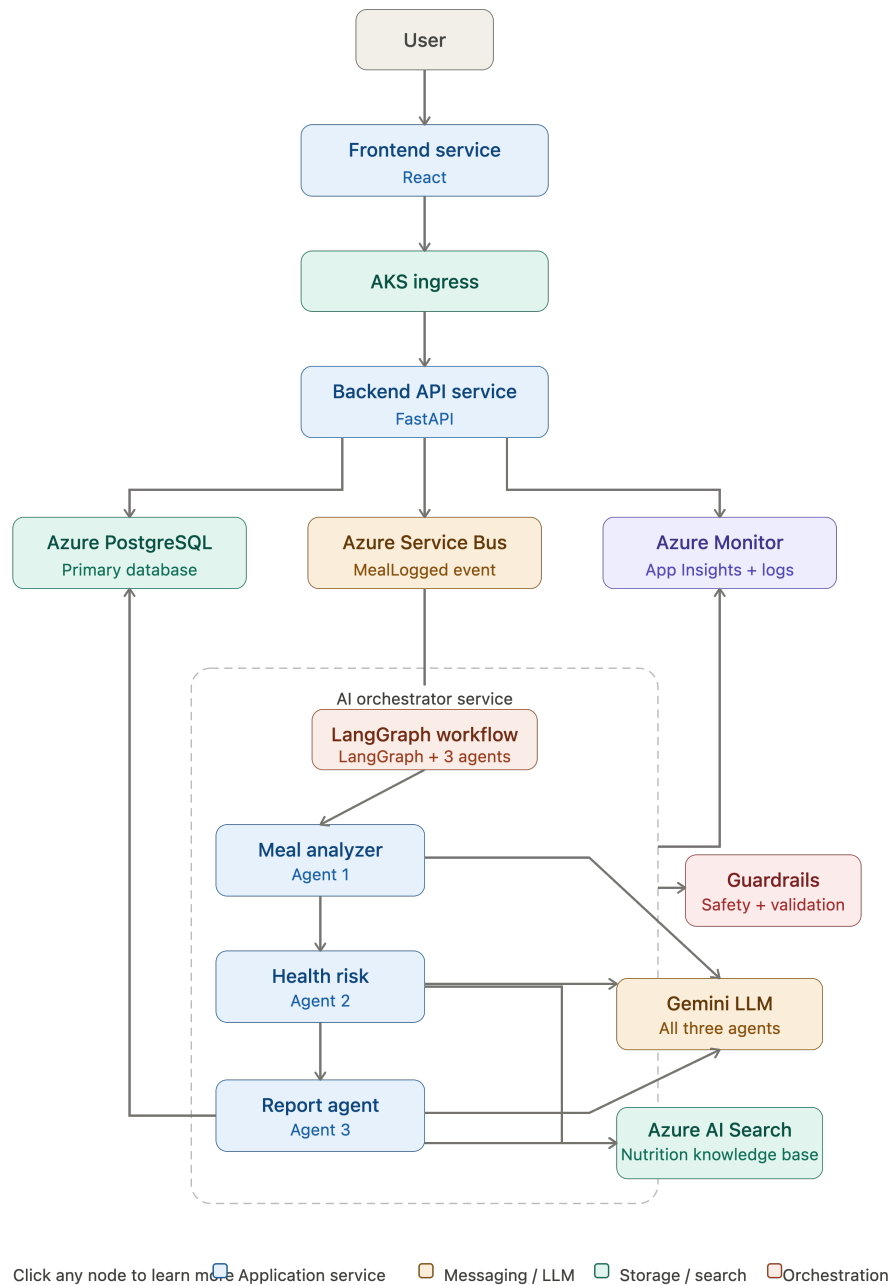


Figure 1: NutriGuard AI — High-Level Architecture Diagram

5.1 Architecture Overview

NutriGuard AI follows an event-driven microservices architecture with three main services:

Service	Responsibility	Technology
Frontend Service	User meal input, profile setup, report display	React
Backend API Service	User data, meal log storage, event publishing	FastAPI + PostgreSQL + Azure Service Bus
AI Orchestrator Service	LangGraph-based 3-agent pipeline execution	LangGraph + Gemini + Azure AI Search

5.2 Event-Driven Flow

The end-to-end flow is:

- 1. User enters meal text and health profile in the React frontend.
- 2. FastAPI backend saves the meal log to PostgreSQL and publishes a MealLogged event to Azure Service Bus.
- 3. The AI Orchestrator consumes the event and triggers the LangGraph agent pipeline.
- 4. Agent 1 (Meal Analyzer) extracts structured food data.
- 5. Agent 2 (Health Risk) retrieves knowledge from Azure AI Search and generates risk flags.
- 6. Agent 3 (Report) composes the final report and writes it back to PostgreSQL.
- 7. The frontend polls or receives a push notification and displays the daily report to the user.

Architecture One-Liner

NutriGuard AI is an event-driven multi-agent nutrition recommender where FastAPI publishes meal events to Azure Service Bus, a LangGraph-based AI Orchestrator runs 3 agents using Gemini and Azure AI Search, PostgreSQL stores the results, and Azure Monitor provides centralized observability.

6. Tech Stack & Azure Services

6.1 Tech Stack

Layer	Technology
Frontend	React
Backend API	FastAPI (Python)
Agent Orchestration	LangGraph
Agent Implementation	Python
LLM	Gemini API (google-generativeai)
RAG / Vector Search	Azure AI Search
Database	PostgreSQL (Azure Database for PostgreSQL)
Event / Queue	Azure Service Bus
Containerization	Docker
Hosting	Azure Kubernetes Service (AKS)
Ingress	AKS Ingress Controller
CI/CD	GitHub Actions
Container Registry	Azure Container Registry
Monitoring	Azure Monitor + Application Insights
Logging	Log Analytics Workspace
Secrets Management	Azure Key Vault
Autoscaling	KEDA on AKS
Guardrails	Safety Prompts + JSON Validation + Azure AI Content Safety (optional)

6.2 Azure Services Mapping

Requirement	Azure Service
Host frontend, backend, and AI Orchestrator	Azure Kubernetes Service (AKS)
Route traffic to services	AKS Ingress Controller
Store Docker images	Azure Container Registry
Store users, profiles, meals, reports	Azure Database for PostgreSQL

Event-driven communication between services	Azure Service Bus
RAG knowledge base for nutrition facts	Azure AI Search
Store secrets and API keys securely	Azure Key Vault
Automate CI/CD deployment pipeline	GitHub Actions
Centralized monitoring and alerting	Azure Monitor
Application-level tracing	Application Insights
Central log storage	Log Analytics Workspace
Monitoring dashboards	Azure Monitor Workbook
Event-based autoscaling of AI Orchestrator	KEDA on AKS
Input/output content safety checks	Azure AI Content Safety (optional)

7. Agent Design

7.1 Agent 1 — Meal Analyzer Agent

Type: Reasoning Agent

Responsibility: Parse the user's natural language meal text and extract structured food item data.

Input	Output (JSON)
'I ate poha with tea and a banana.'	{ "foods": ["poha", "tea", "banana"], "carb_sources": ["poha", "banana"], "protein_sources": [], "beverages": ["tea"] }

Key reasoning tasks:

- Identify food items from informal Indian meal descriptions.
- Classify food into macro categories: carbohydrate, protein, fat, beverage, fruit/vegetable.
- Flag missing food groups (e.g., no protein source at breakfast).
- Validate JSON output before passing to the next agent.

7.2 Agent 2 — Health Risk & Supplement Agent

Type: Reasoning + Retrieval Agent

Responsibility: Cross-check the extracted meal against the user's health condition and goal using trusted nutrition knowledge from Azure AI Search.

Input	Output (JSON)
Meal: poha + tea Condition: iron deficiency	{ "risk_flags": [{ "type": "iron_attention", "severity": "medium", "message": "Tea near meals may need attention for iron deficiency."}] }

Knowledge base documents (stored in Azure AI Search):

- Iron deficiency guidance and dietary sources
- Tea and tannins — notes on iron absorption
- Vegetarian protein sources for Indian diet
- Fat loss nutrition basics — protein and calorie balance
- Vitamin B12 sources and deficiency notes
- Vitamin D sources and supplementation notes
- Supplement safety notes and interaction flags

7.3 Agent 3 — Report Agent

Type: Acting Agent

Responsibility: Compose the final safe daily nutrition report. Convert risk flags and suggestions into simple, safe language. Always include the safety disclaimer.

Report structure:

- Meal summary: 'Today's breakfast was carbohydrate-heavy with low protein.'
- Risk flags in plain language with medium/low severity labels.
- Actionable suggestions with real Indian food alternatives.
- Safety note: Always appended for any health condition user.

8. Data Sources & Schema

8.1 User Data

Stored in PostgreSQL. Collected at onboarding and updated by the user.

Table	Fields
users	user_id, name, email, created_at
user_profiles	profile_id, user_id, health_goal, diet_type, health_conditions[], supplements[]
meal_logs	meal_id, user_id, meal_text, meal_time, date, processed
nutrition_flags	flag_id, meal_id, flag_type, severity, message
daily_reports	report_id, user_id, date, report_text, generated_at
outbox_events	event_id, event_type, payload, published, created_at

8.2 Knowledge Base Data (Azure AI Search)

Pre-loaded curated nutrition documents. Retrieved by Agent 2 via semantic search.

Document	Content Summary
iron_deficiency_guide	Dietary iron sources (heme and non-heme); absorption enhancers (Vitamin C); absorption inhibitors (tannins in tea)
tea_iron_absorption	Evidence notes on tannins in tea and their effect on non-heme iron absorption; timing recommendations
vegetarian_protein_sources	Indian vegetarian protein sources: dal, paneer, tofu, soy chunks, chana, sprouts, curd, eggs
fat_loss_nutrition	Protein targets for fat loss; caloric balance; importance of satiety foods
vitamin_b12_notes	B12 sources for vegetarians; deficiency symptoms; supplement safety notes
vitamin_d_notes	Vitamin D food sources; sunlight exposure; supplement safety notes
supplement_safety	General supplement interaction notes; always refer to doctor for dosage

9. Guardrails & Safety

9.1 Safety Rules

NutriGuard AI is built with strict safety guardrails to ensure it never crosses into medical territory:

Rule	Implementation
No medical diagnosis	System prompt explicitly forbids diagnosis language; output validated pre-delivery
No medicine dosage recommendation	Agent 2 and Report Agent prompted to redirect all dosage questions to doctor
No supplement dose changes	Knowledge base entries include 'consult doctor' notes; prompt enforces this
Always add safety disclaimer	Report Agent always appends the safety note for any health condition user
JSON output validation	All agent JSON outputs are schema-validated before passing to the next agent
Block overconfident medical claims	Prompt rules and keyword filters catch terms like 'cures', 'treats', 'proven to fix'
Optional: Azure AI Content Safety	Input and output can be passed through Azure AI Content Safety API for production use

9.2 Standard Safety Disclaimer

Safety Note (always included in reports)

This is general nutrition guidance only. NutriGuard AI is not a medical tool. For supplements, health conditions, or medication, always follow your doctor's advice.

10. Observability & Monitoring

10.1 Correlation ID Strategy

Every request is tagged with a common `correlation_id = meal_log_id`. This ID flows through all services and agents, enabling full end-to-end tracing in Log Analytics Workspace.

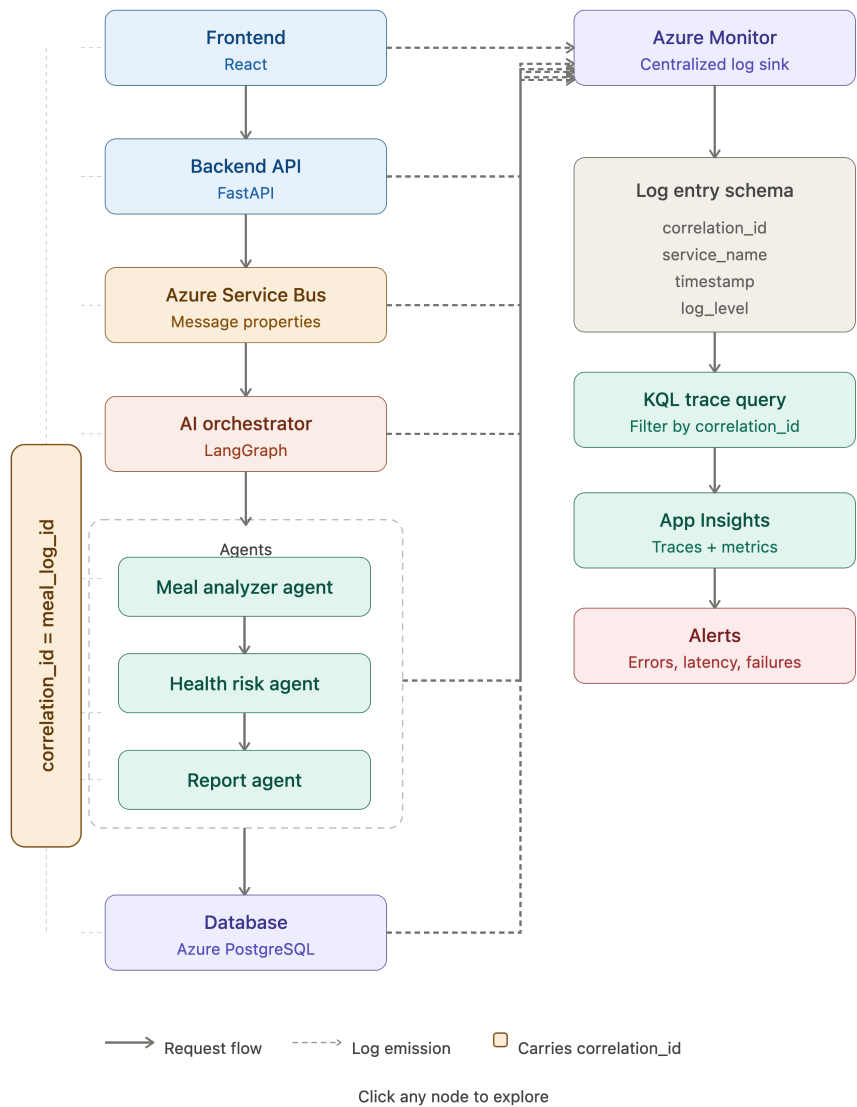


Figure 2: NutriGuard AI — Observability Plan and Correlation ID Flow

10.2 Monitored Metrics

Metric	Tool
--------	------

API endpoint latency (p50, p95)	Application Insights
AI Orchestrator end-to-end latency	Application Insights + custom spans
Agent-wise latency (per agent step)	LangGraph trace + Application Insights
Azure Service Bus queue depth	Azure Monitor
Failed or dead-lettered events	Azure Monitor + alerts
Gemini API errors and quota exhaustion	Application Insights exceptions
Azure AI Search query errors	Azure Monitor diagnostics
PostgreSQL query latency and errors	Azure Database metrics
AKS pod restarts and OOM events	Azure Monitor container metrics
End-to-end report generation time	Custom Azure Monitor metric

11. Success Metrics

Metric	Target
Food item extraction accuracy	> 90%
Risk flag accuracy	> 85%
Agent JSON output validity rate	> 95%
Safety violation rate (diagnosis / dosage)	0%
End-to-end report generation latency (demo)	< 20 seconds
Failed workflow rate	< 5%
Correlation ID trace coverage	100% of requests

12. Scope & Out-of-Scope

12.1 In Scope (MVP)

- Simple natural language meal text input
- User health profile: goal, diet type, conditions, supplements
- 3-agent LangGraph pipeline (Meal Analyzer → Health Risk → Report)
- Safe risk flags with medium/low severity
- Daily nutrition report with suggestions and disclaimer
- Event-driven architecture using Azure Service Bus
- Local development with PostgreSQL
- Azure-ready deployment architecture
- Centralized logging with correlation_id tracing

12.2 Out of Scope (MVP)

- Exact calorie or macro gram calculation
- Medical diagnosis of any health condition
- Medicine or prescription dosage recommendations
- Changes to existing prescriptions
- Image-based food recognition
- Wearable device integration (e.g., Fitbit, Apple Watch)
- Full mobile application (iOS/Android)
- Real doctor consultation or teleconsultation workflow